

I. Introduction

Dans le domaine du deep learning, les réseaux de neurones (NN) se sont imposés comme des outils puissants pour résoudre des problèmes complexes tels que la vision par ordinateur, la détection d'objets, la reconnaissance d'images, l'analyse prédictive et le traitement du langage naturel.

Cependant, les performances et l'efficacité d'un réseau de neurones ne dépendent pas uniquement de son architecture ou du volume de données utilisées. Un facteur déterminant de son succès réside dans le choix et le réglage des hyperparamètres.

Contrairement aux paramètres du modèle (poids et biais), qui sont appris automatiquement pendant l'entraînement, les hyperparamètres sont définis avant le processus d'apprentissage. Ils contrôlent le comportement du modèle, influencent la vitesse de convergence, la précision ainsi que la capacité de généralisation.

On distingue principalement deux catégories d'hyperparamètres :

- Hyper-paramètres d'optimisation
- Hyper-paramètres d'architecture

La régularisation est également essentielle pour éviter le surapprentissage (*overfitting*).

❖ Rôle du réglage des hyperparamètres

Le réglage des hyperparamètres consiste à identifier les meilleures valeurs des paramètres de configuration afin d'optimiser les performances d'un modèle de machine learning. Ce processus est expérimental : plusieurs combinaisons sont testées jusqu'à trouver celle qui minimise la fonction de perte et améliore la précision du modèle.

Les hyperparamètres influencent directement l'apprentissage du modèle, notamment sa capacité à généraliser et à éviter le surapprentissage. Un bon réglage permet ainsi de trouver un équilibre entre biais et variance.

On peut comparer ce processus à un orchestre : chaque hyperparamètre joue un rôle spécifique, et seule une bonne coordination permet d'obtenir une performance harmonieuse du modèle.

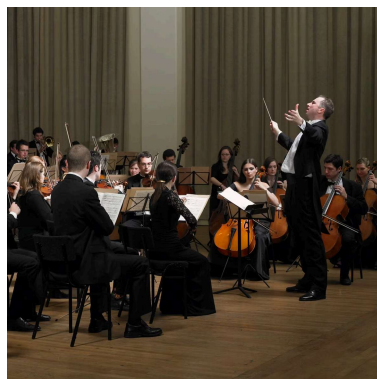


Figure 1 : Analogie du réglage des hyperparamètres (orchestre)

1. Hyper-paramètres d'optimisation

Ces hyper-paramètres contrôlent comment le modèle apprend.

1.1 Taux d'apprentissage (Learning Rate)

- Détermine la taille des pas lors de la mise à jour des poids.
- Trop grand → divergence
- Trop petit → apprentissage lent

Bonnes pratiques (IBM) :

- Utiliser des schedulers (decay, cosine annealing)
- Commencer avec 0.001 (Adam)

1.2 Nombre d'itérations / époques

- Une époque = passage complet sur les données
- Trop d'époques → overfitting

Solution :

- Early Stopping

1.3 Batch size

- Nombre d'exemples traités en une fois

Batch size	Effet
Petit	+ bruit, meilleure généralisation
Grand	+ rapide, mais peut overfit

1.4 Optimiseur

Méthode utilisée pour minimiser la fonction de coût :

- SGD : simple mais lent
- Adam : rapide et adaptatif (le plus utilisé)
- RMSProp : adapté aux séries temporelles

Recommandation :

- Adam par défaut

1.5 Fonction de coût (Loss Function)

Exemples :

- Classification $\rightarrow$ Cross-Entropy
- Régression $\rightarrow$ MSE

1.6 Régularisation (λ)

L1 (Lasso)

- Favorise la sparsité (poids = 0)

L2 (Ridge)

- Réduit les poids sans les annuler

Objectif :

- Éviter l'overfitting

2. Hyper-paramètres d'architecture

Ces paramètres définissent la structure du réseau.

2.1 Nombre de couches

- Peu de couches $\rightarrow$ underfitting
- Trop $\rightarrow$ overfitting + complexité

2.2 Nombre de neurones

- Plus de neurones = plus de capacité
- Mais risque de surapprentissage

2.3 Type de couches

- FC (Fully Connected) → classique
- CNN → images
- RNN / LSTM / GRU → données séquentielles

2.4 Fonction d'activation

- ReLU (la plus utilisée)
- Sigmoid (classification binaire)
- Tanh

IBM recommande ReLU pour éviter le vanishing gradient

2.5 Dropout

- Désactive aléatoirement des neurones
- Permet de réduire l'overfitting

Valeur typique : 0.3 – 0.5

2.6 Paramètres spécifiques CNN

- ♦ Taille du filtre (kernel size)
 - Détecte des motifs locaux
- ♦ Stride
 - Pas de déplacement du filtre
- ♦ Padding
 - Ajout de bordures pour conserver la taille

2.7 Pooling

- MaxPooling → garde info importante
- AvgPooling → moyenne

2.8 Architectures avancées

- ResNet
- VGG
- Transfer Learning

Permet de réutiliser des modèles pré-entraînés

3. Régularisation (approfondissement)

Techniques pour améliorer la généralisation :

- ✓ Dropout
- ✓ L1 / L2
- ✓ Early stopping
- ✓ Data augmentation

4. Stratégies de choix des hyper-paramètres

◆ Grid Search

- Teste toutes les combinaisons

◆ Random Search

- Plus efficace

◆ Bayesian Optimization

- Intelligent et optimisé

Ces méthodes permettent d'explorer efficacement l'espace des hyperparamètres afin d'obtenir de meilleures performances.

Conclusion

Les hyper-paramètres jouent un rôle essentiel dans la performance des modèles de deep learning. Une bonne combinaison permet :

- Une convergence rapide
- Une meilleure précision
- Une bonne généralisation

Les pratiques modernes recommandent l'utilisation d'optimiseurs comme Adam, de techniques de régularisation et de méthodes automatiques pour le tuning.

Les hyperparamètres constituent un élément clé dans la performance des modèles de deep learning. Leur bon réglage permet d'obtenir un modèle plus stable, précis et généralisable. L'utilisation de techniques modernes comme le tuning automatique et les méthodes d'optimisation améliore considérablement les résultats des réseaux de neurones.

Ressources

- IBM Deep Learning Guide
<https://www.ibm.com/cloud/learn/deep-learning>
- Deep Learning Book – Ian Goodfellow
<https://www.deeplearningbook.org/>
- Coursera – Deep Learning Specialization (Andrew Ng)
<https://www.coursera.org/specializations/deep-learning>
- Documentation TensorFlow
<https://www.tensorflow.org/learn>
- Documentation PyTorch
<https://pytorch.org/docs/stable/index.html>
- Papers with Code
<https://paperswithcode.com/>